Encorely · Requisitos de despliegue en Azure

Oct 29, 2026 · @Juan

Encorely se despliega en Azure con **2 servidores Ubuntu Server 24.04 LTS** (aplicaciones y base de datos) detrás de un **Azure Application Gateway**. Este documento reemplaza la propuesta de Render del manual ENC-MT-001 v1.4 y llena las secciones 3.2.3, 3.3 y 3.4, que hoy están vacías.

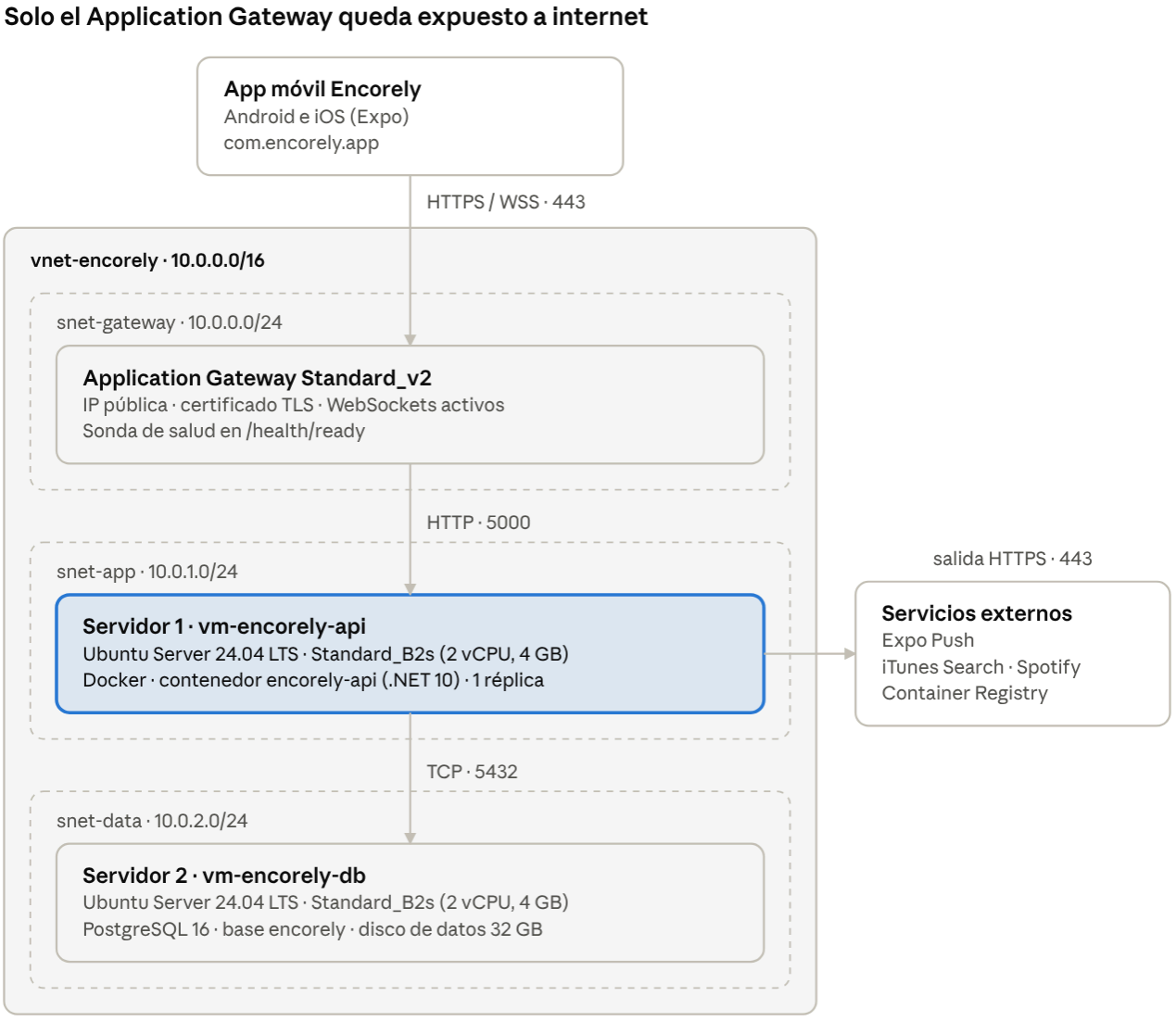
1. Contexto y alcance

Aspecto	Manual v1.4	Este documento
Proveedor	Render (servicio web + Postgres administrado)	Microsoft Azure
Ambientes cubiertos	Staging y Producción propuestos, no creados	Staging y Producción en Azure
Réplicas de la API	1, el balanceador no reparte carga	1, se mantiene
Base de datos	PostgreSQL 16, sin extensiones	PostgreSQL 16, sin extensiones

Se conserva todo lo que el manual ya define: .NET 10, PostgreSQL 16, 25 tablas, puerto 5432, health checks y variables de entorno. Lo que cambia es dónde corre cada nodo.

Los tamaños de las máquinas son una **propuesta del equipo para el piloto**; el manual no los define y se ajustarán con el uso real.

2. Arquitectura de despliegue en Azure



despliegue en Azure · 2 servidores, 1 balanceador, 3 subredes

El tráfico entra solo por el Application Gateway; la API y la base quedan en subredes privadas y cada una habla solo con la siguiente.

3. Requisitos de hardware (3.2.3 Servidores de producción)

Cada servidor es una máquina virtual de Azure en la región **East US 2** o **Brazil South**; las dos VM, la base y el balanceador van en la misma región.

Servidor	Nombre del recurso	Tamaño de VM	vCPU	RAM	Disco del SO	Disco de datos
1. Aplicaciones	`vm-encorely-api`	Standard_B2s	2	4 GB	30 GB SSD Premium	No aplica

Servidor	Nombre del recurso	Tamaño de VM	vCPU	RAM	Disco del SO	Disco de datos
2. Base de datos	`vm-encorely-db`	Standard_B2s	2	4 GB	30 GB SSD Premium	32 GB SSD Premium, montado en `/var/lib/postgresql`
Balanceador	`agw-encorely`	Application Gateway Standard_v2	Gestionado	Gestionado	Gestionado	Gestionado

El disco de datos va separado del disco del SO para poder ampliarlo, respaldarlo y restaurarlo sin tocar el sistema operativo.

Alternativa gestionada para el servidor 2: Azure Database for PostgreSQL Flexible Server, nivel Burstable B1ms (1 vCPU, 2 GB RAM, 32 GB). Azure administra el SO, los parches y los respaldos.

3.1 Requisitos de las máquinas virtuales

Estos son los valores que se eligen al crear cada VM en el portal de Azure. Aplican igual a los dos servidores.

Opción al crear la VM	Valor
Imagen	Ubuntu Server 24.04 LTS de Canonical, x64, Generación 2
Arquitectura	x64
Tamaño	Standard_B2s (2 vCPU, 4 GB RAM)
Autenticación	Llave pública SSH, sin contraseña
IP pública	Ninguna: solo IP privada dentro de su subred
Disco del SO	30 GB SSD Premium
Identidad administrada	Activada en el servidor 1, con permiso de lectura en Container Registry y Key Vault
Respaldo	Azure Backup diario de la VM (servidor 2 como mínimo)
Apagado automático	Solo en Staging, para no pagar horas sin uso

La virtualización la pone Azure: no se instala ningún hipervisor. Cada VM se crea en la subred indicada en la sección 5.

4. Requisitos de software (3.3)

4.1 Servidor 1: aplicaciones

Software	Versión	Para qué
Ubuntu Server	24.04 LTS	Sistema operativo
Docker Engine	Última estable	Corre el contenedor `encorely-api`
docker compose	v2	Levanta el contenedor con su configuración
Imagen `encorely-api`	La del release	API ASP.NET Core 10 construida con el Dockerfile de `encorly_back`
Azure CLI	Última estable	Login al Azure Container Registry y lectura de Key Vault

No se instala el SDK ni el runtime de .NET en la VM: ya viene dentro de la imagen.

4.2 Servidor 2: base de datos

Software	Versión	Para qué
Ubuntu Server	24.04 LTS	Sistema operativo
PostgreSQL	16	Motor de la base `encorely` (25 tablas, sin extensiones)
pg_dump / pg_restore	16	Respaldo y restauración

4.3 Servicios de Azure

Servicio	Para qué
Application Gateway	Balanceador, HTTPS y WebSockets hacia la API
Azure Container Registry	Guarda las imágenes de la API por versión
Azure Key Vault	Guarda contraseñas y llaves de la API
Azure Monitor	Métricas y alertas de las VM

4.4 Por qué Ubuntu y no Windows Server

- **.NET 10 es multiplataforma.** Solo el .NET Framework 4.x antiguo exige Windows.
- **La API se distribuye como contenedor Linux.** En Windows harían falta contenedores Windows o reescribir el despliegue para IIS.
- **Es el mismo SO del CI.** GitHub Actions ya compila y prueba en Ubuntu 24.04.
- **Menor costo y consumo.** Una VM Windows Server paga licencia y usa más RAM solo para el sistema.

4.5 Qué hay que instalar en cada servidor

Todo se instala sobre Ubuntu Server 24.04 LTS recién creado, con un usuario con `sudo` y acceso por SSH.

Servidor 1, aplicaciones

#	Qué se instala	Para qué
1	Actualizaciones del sistema	Parches de seguridad al día
2	Docker Engine y docker compose v2 (repositorio oficial de Docker)	Correr el contenedor de la API
3	Azure CLI	Descargar la imagen de Container Registry y leer Key Vault
4	Imagen `encorely-api` del release	La API en sí

```
# 1. Actualizar el sistema
sudo apt update && sudo apt upgrade -y

# 2. Docker Engine + compose v2
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(.
/etc/os-release && echo $VERSION_CODENAME) stable" | sudo tee
/etc/apt/sources.list.d/docker.list
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-
plugin
sudo systemctl enable --now docker

# 3. Azure CLI
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash

# 4. Descargar la imagen del release
az login --identity
az acr login --name <registro>
docker pull <registro>.azurecr.io/encorely-api:<version>
```

Servidor 2, base de datos

#	Qué se instala o configura	Para qué
1	Actualizaciones del sistema	Parches de seguridad al día
2	Disco de datos formateado y montado en `/var/lib/postgresql`	Separar los datos del SO
3	PostgreSQL 16 (viene en los repositorios de Ubuntu 24.04)	Motor de la base
4	`listen_addresses` y `pg_hba.conf`	Escuchar solo en la IP privada y aceptar solo a `snet-app`
5	Base `encorely` y usuario `encorely_app`	La app no usa el superusuario `postgres`

```
# 1. Actualizar el sistema
sudo apt update && sudo apt upgrade -y

# 3. PostgreSQL 16 (instalar después de montar el disco de datos)
sudo apt install -y postgresql-16

# 4. Red: en /etc/postgresql/16/main/postgresql.conf
#    listen_addresses = '10.0.2.4'
#    y al final de /etc/postgresql/16/main/pg_hba.conf
#    hostssl  encorely  encorely_app  10.0.1.0/24  scram-sha-256
sudo systemctl restart postgresql

# 5. Base y usuario de la app
sudo -u postgres psql -c "CREATE USER encorely_app WITH PASSWORD '<desde Key
Vault>';"
sudo -u postgres psql -c "CREATE DATABASE encorely OWNER encorely_app;"
```

La base no necesita extensiones de PostgreSQL. Las tablas no se crean a mano: las crean las 11 migraciones de EF Core (sección 7, paso 5).

5. Requisitos de red (3.4)

Todo vive en una red virtual `vnet-encorely` (10.0.0.0/16) con tres subredes. Solo el Application Gateway tiene IP pública; las VM quedan en IP privada.

Subred	Rango	Qué hay	Grupo de seguridad (NSG)
`snet-gateway`	10.0.0.0/24	Application Gateway	`nsg-gateway`
`snet-app`	10.0.1.0/24	Servidor 1, aplicaciones	`nsg-app`
`snet-data`	10.0.2.0/24	Servidor 2, base de datos	`nsg-data`

5.1 Reglas de puertos

Origen	Destino	Puerto	Protocolo	Para qué
Internet (app móvil)	Application Gateway	443	HTTPS / WSS	API REST `/api/v1` y chat `/hubs/chat`
Application Gateway	Servidor 1	5000	HTTP	Tráfico hacia la API y sonda `/health/ready`

Origen	Destino	Puerto	Protocolo	Para qué
Servidor 1	Servidor 2	5432	TCP	Conexión de la API a PostgreSQL
IP del equipo	Servidor 1 y 2	22	SSH	Administración, solo desde IPs autorizadas
Servidor 1	Internet	443	HTTPS	Expo Push, iTunes Search, Spotify y Container Registry

Todo lo que no está en la tabla queda **denegado**. El puerto 5432 nunca se abre a internet.

5.2 Dominio y certificado

- Dominio de la API, por ejemplo `api.encorely.co` (por definir), apuntando a la IP pública del Application Gateway.
- Certificado TLS en el Application Gateway, guardado en Key Vault.
- WebSockets para SignalR: el Application Gateway los soporta siempre y no se pueden apagar (Microsoft). Solo hay que poner el tiempo de espera del backend por encima del intervalo de ping del chat.

5.3 Ancho de banda

Encorely mueve poco tráfico por sus servidores: la API responde JSON pequeño y el chat envía mensajes cifrados de pocos KB. Las portadas y las muestras de 30 s salen de iTunes directo al teléfono, sin pasar por Azure.

Tramo	Ancho de banda mínimo propuesto	Por qué
Teléfono del usuario	4G o wifi	Muestras de audio de iTunes y chat en tiempo real
Internet ↔ Application Gateway	10 Mbps sostenidos	Tráfico de la API y del chat en el piloto
Application Gateway ↔ Servidor 1	100 Mbps	Red interna de Azure, misma región
Servidor 1 ↔ Servidor 2	100 Mbps	Consultas SQL dentro de la misma VNet
Servidor 1 → Internet	10 Mbps	Avisos push de Expo y consultas a iTunes y Spotify

Estas cifras son una estimación del equipo para el piloto. Microsoft no publica un ancho de banda garantizado para la serie B (Microsoft Q&A), así que se validan midiendo:

- Entre las dos VM: `iperf3 -s` en el servidor 2 y `iperf3 -c 10.0.2.4 -t 30` en el servidor 1.
- En producción: métricas *Network In* y *Network Out* de cada VM en Azure Monitor.

La salida de datos a internet tiene costo en Azure. Por eso conviene que audio e imágenes sigan saliendo de iTunes y no de los servidores.

6. Configuración de la API

La API no arranca sin estas variables. Los valores secretos se guardan en Key Vault `kv-encorely` y nunca en archivos versionados, como exige el manual en 6.1.

Variable	Valor en Azure	¿Secreto?
`ASPNETCORE_ENVIRONMENT`	`Production`	No
`DB_HOST`	IP privada del servidor 2 (por ejemplo 10.0.2.4)	No
`DB_PORT`	`5432`	No
`DB_NAME`	`encorely`	No
`DB_USER`	Usuario propio de la app, no `postgres`	No
`DB_PASSWORD`	Contraseña del usuario de la app	Sí
`DB_SSL_MODE`	`Require`	No
`BANNED_EMAIL_KEY`	Llave HMAC del veto de correos	Sí
Llave del JWT	Llave HS256 de los access tokens	Sí

Pendiente: el manual no da el nombre exacto de la variable del JWT, porque la sección 5.3.1 está vacía. Hay que confirmarlo en el código de `encorly_back`.

En producción no se definen `DEMO_USER_PASSWORD` ni `SEED_REVIEWER_ONLY`, para no sembrar cuentas demo.

7. Orden de arranque y validación

Los servidores se levantan de atrás hacia adelante: primero la base, luego la API y al final el balanceador.

1. **Red:** crear `vnet-encorely`, las tres subredes y los NSG con las reglas de 5.1.

2. **Servidor 2:** instalar PostgreSQL 16, crear la base `encorely` y el usuario de la app, y abrir 5432 solo a `snet-app`.
3. **Secretos:** cargar `DB_PASSWORD`, `BANNED_EMAIL_KEY` y la llave del JWT en Key Vault.
4. **Imagen:** GitHub Actions construye la imagen con el Dockerfile y la sube a Azure Container Registry con la etiqueta del release.
5. **Migraciones:** aplicar las 11 migraciones de EF Core contra la base antes del primer arranque. En producción no se aplican solas como en Development.
6. **Servidor 1:** instalar Docker, descargar la imagen y arrancar **un solo contenedor** con las variables de la sección 6. Una sola réplica, porque el bloqueo por intentos de login vive en memoria.
7. **Balanceador:** configurar el Application Gateway hacia el servidor 1 en el puerto 5000, con certificado, WebSockets y sonda en `/health/ready`.
8. **App móvil:** compilar con EAS apuntando a la URL pública de la API.

7.1 Criterios de validación

Prueba	Resultado esperado
`GET /health/live`	`Healthy`
`GET /health/ready`	`Healthy` (confirma que la API llega a la base)
`GET /openapi/v1.json`	Título `Encorely API`
Login desde la app	Responde `accessToken`, `refreshToken`, `expiresIn: 900` y `user`
Chat entre dos cuentas	El mensaje llega por `/hubs/chat` a través del Application Gateway
Puerto 5432 desde internet	Sin respuesta

8. Cambios en el manual ENC-MT-001

Con este cambio de proveedor el manual pasa a la versión 1.5, y el cambio se registra en el historial de 1.2.

Sección	Dice hoy	Debe decir
3.1 Ambientes	Staging y Producción en Render	Staging y Producción en Azure

Sección	Dice hoy	Debe decir
3.2.3 Servidores de producción	Vacía	Tabla de la sección 3 de este documento
3.3 Requisitos de software	Vacía	Tablas de la sección 4
3.4 Requisitos de red	Vacía	Subredes y puertos de la sección 5
5.3.1 Variables de la API	Vacía	Tabla de la sección 6
5.5.4 Despliegue de la API	Basado en `render.yaml`	Imagen en Container Registry y arranque en el servidor 1 (sección 7)
5.8.1 Diagrama de despliegue	Nodos de Render	Diagrama de la sección 2 de este documento
6.1 Motor	Producción propuesta: Render PostgreSQL 16	PostgreSQL 16 en Azure
8.1 Perfiles técnicos	Acceso al panel de Render	Acceso al portal de Azure con roles mínimos y dos factores

Preguntas abiertas

- [ ] ¿El profesor exige VMs o acepta servicios gestionados como Flexible Server?
- [ ] ¿Cuál es el nombre exacto de la variable de la llave del JWT?
- [ ] ¿Qué dominio usará la API?